

Directivas del ensamblador

Las instrucciones que podemos utilizar con un dispositivo son las que proporciona el fabricante para su producto y que forman parte del llamado "*repertorio de instrucciones*". Pero al utilizar un programa ensamblador podemos introducir además instrucciones o comando que proporciona el propio ensamblador. Estos comandos generalmente se utilizan para simplificar la tarea de programar, y reciben el nombre de directivas.

Por lo tanto las directivas no se traducen directamente a instrucciones del lenguaje máquina sino que asignan al programa ciertas áreas de memoria, definen símbolos, designan áreas de RAM para almacenamiento de datos temporales, colocan tablas o datos constantes en memoria y permiten referencias a otros programas.

Las directivas se utilizan como comandos escritos en el código fuente para realizar un control directo o ahorrar tiempo a la hora de ensamblar. El resultado de incorporar directivas se puede ver en el fichero *.LST, después de ensamblar el programa.

Para usar éstas directivas o pseudo-operandos, el programador las coloca en el campo del código de operación, y, si lo requiere la directiva, una dirección o dato en el campo de dirección.

Hay que aclarar que las instrucciones de los PIC's son únicas y que no hay nada mas, por ejemplo en el PIC16F84A son sólo 35 (ver instrucciones del PIC16F84A). Esto debe tenerse claro porque cuando se comienza con el ensamblador pueden confundirse un poco las propias instrucciones de los PIC's con las directivas propias del ensamblador.

A continuación se exponen las más relevantes.

Directiva EQU

El nombre viene de la palabra "equal", (igual)". La directiva EQU permite al programador "igualar" nombres personalizados a datos o direcciones. Los nombres utilizados se refieren generalmente a direcciones de dispositivos, datos numéricos, direcciones de comienzo, direcciones fijas, posiciones de bits, etc. Un nombre es más descriptivo que una simple dirección y la tarea de programar se hará mucho más sencilla. También podemos asignar un nombre a una instrucción que repitamos varias veces a lo largo de un algoritmo, de manera que sea mucho más sencilla la programación. A estos nombre que asignamos mediante esta directiva se les denomina constantes, ya que el registro al que apuntan no variará durante el programa

Ejemplos:

```
temp    equ    12
DATO    EQU    22
PORT_A  EQU    5
START   EQU    0
CARRY   EQU    3
TIEMPO  EQU    5
Bank_1  EQU    BSF STATUS,RP0
```

Estas líneas también pueden están incluidas en un archivo aparte al ASM ([véase directiva INCLUDE](#)).

No siempre es necesario que con esta directiva se igualen posiciones de memoria a las etiquetas, ya que podemos poner nombres a datos. Podemos definir una equivalencia con el nombre de otra equivalencia ya definida y rtealizar operaciones matemáticas. Por ejemplo, podemos calcular la frecuencia del ciclo máquina a partir de la frecuencia de reloj con la finalidad de emplearla para hacer otros cálculos de la manera que se describe a continuación:

```
PORT_B   EQU    PORT_A+1
PORT_C   EQU    PORT_A+2
FIN      EQU    START+100
FIN2     EQU    START+200
clockrate EQU    .4000000    ;frecuencia del cristal
fclk     EQU    clockrate/4   ;frecuencia del reloj interno
```

El valor del operando debe estar ya definido anteriormente, sino el compilador entregará un error.

Además de esto, podemos igualar a las etiquetas cualquier otro tipo de valores que usemos, como, por ejemplo, el cero y el 1 en el bit de destino:

W	EQU	0
F	EQU	1

Con esto último, cuando usemos una instrucción en donde debamos especificar donde se almacenará el resultado, en **w** o en un registro, en lugar de escribir :

- 1: para que el resultado se almacene en **f**.
- 0: para que el resultado se almacene en **w**.

Pondremos:

- F: para que el resultado se almacene en **f**.
- W: para que el resultado se almacene en **w**.

Generalmente esto último no será necesario realizarlo, siempre que incluyamos el fichero "INC" correspondiente al PIC con el que estemos trabajando ([véase directiva INCLUDE](#)).

Directiva ORG

Esta directiva dice al ensamblador a partir de que posición de memoria de programa se situarán las siguientes instrucciones. Rutinas de comienzo, subrutinas de interrupción y otros programas deben comenzar en locaciones de memoria fijados por la estructura del microcontrolador. Recordemos que el 16F84 sólo tiene 1024 posiciones de memoria flash para código.

La directiva ORG hace al compilador colocar el código que le sigue en una nueva dirección de memoria (la salida del compilador no solo coloca los códigos de operación sino también las direcciones de cada instrucción del programa). Usualmente se la utiliza para: reset, programas de servicios de interrupción, programa principal, subrutinas.

Ejemplos:

1) Inicia el programa en la posición cero:

```
ORG    0x00
```

2) Inicia el programa en la posición 0000h y luego pasa a la 0005h para no utilizar la posición del vector de interrupción (0004 h)

```
ORG    0x00    ; El programa comienza en la dirección 0 y
GOTO    inicio ; salta a la dirección 5 para sobrepasar
ORG    0x05    ; el vector de interrupción, situado en la
posición 4
Inicio xxx...
```

3) Inicia el programa en la posición 0000h y luego pasa a la 0005h para no utilizar la posición del vector de interrupción (0004 h). Si se produce una interrupción se pasa a la posición **interr**. Las subrutinas comienzan a partir de la dirección 0300h.

```
ORG    00h      ;vector de reset
goto    inicializa
ORG    04h      ;vector de interrupción
goto    interr
ORG    05h
inicializa movlw 08h      ;aquí comienza el programa
.
.
ORG    300h     ;subrutinas
Subrutina1
.
.
return
Subrutina2
.
.
return
```

Directiva #INCLUDE

Esta directiva indica que archivos deberán tomarse en cuenta a la hora de compilar el código. Normalmente se usa para incluir el archivo de PIC que el ensamblador tiene entre sus archivos, con el cual el compilador será capaz de reconocer todos los

registros especiales y sus bits. Su uso nos recordará al `#include` del lenguaje C. Esta línea debe colocarse al principio, y tiene la siguiente sintaxis:

```
#INCLUDE          ; Lista de etiquetas de microchip
```

En ciertas ocasiones gran cantidad errores son debidos a que el nombre del archivo puesto entre comillas no se escribe correctamente.

Si utilizamos [MPLAB](#), un entorno de desarrollo que proporciona gratuitamente Microchip, dispondremos de los archivos con extension **.INC** para cada uno de los PIC desarrollados hasta la aparición de la versión de **MPLAB** que utilicemos. En estos archivos se definen todos los registros así como otros elementos de acuerdo al microcontrolador que estemos utilizando.

También podemos crear nuestros propios archivos "INC" con funciones, definiciones y subrutinas que utilicemos a menudo en nuestro código para evitar tener que copiarlas cada vez.

El archivo [P16F84A.INC](#) que viene con MPLAB contiene definiciones de registros, bits y bits de configuración. Los archivos INC pueden verse con cualquier editor de texto pero no se recomienda modificarlos, para no perder compatibilidad con programas desarrollados por otros.

Utilizar el **INC** del PIC que estamos utilizando en nuestro programa no es obligatorio, y podemos omitirlo, pero a cambio tendremos que definir los nombres de los registros que usemos o bien llamarlos por su posición de memoria.

Esto puede a la larga ser problemático de manera que se recomienda utilizar los archivos **INC** correspondientes al PIC que utilicemos porque además de facilitar la creación del programa al no tener que recordar las direcciones reales de los registros también se facilita el paso de un programa diseñado para un microcontrolador hacia otro distinto.

Si utilizamos las posiciones de memoria con la dirección real, podemos hacer incompatibles las operaciones entre registros. Por ejemplo, CLRF 0x05, borra el registro ubicado en esa dirección, que no es ni más ni menos que el PORTA (Puerto A) en el PIC16F84A. Pues bien, si queremos actualizarnos a otro microcontrolador pero resulta que en este el registro 0x05 tiene otra función nos será mucho más difícil actualizar el programa. Ahora bien, si hubiésemos utilizado CLRF PORTA, y el .INC correspondiente al nuevo microcontrolador ya se ocupará el ensamblador de realizar las correspondencias.

Y por supuesto siempre será más fácil recordar PORTA que no 0x05.

También permite incluir otros programas. Por ejemplo:

```
#INCLUDE "DISPLAY.ASM"
```

Esto le dice al compilador que incluya el código que se encuentra en el archivo DISPLAY.ASM como si fuese parte del propio programa. Esto es muy útil para reutilizar códigos realizados con anterioridad.

Directiva LIST

Este comando sirve para que el compilador tenga en cuenta sobre qué procesador se está trabajando. Este comando debe estar en todo proyecto, situado debajo del "include", con la siguiente sintaxis.

```
LIST P=PIC16F84A
```

Directiva END

Al igual que las dos anteriores, esta debe ir incluida una sola vez en todo el programa. En concreto, esta debe situarse al final, para indicar al ensamblador que el programa ha finalizado. Esta siempre debe estar presente, aunque el flujo de nuestro programa acabe en un bucle.

Directiva #DEFINE

`#DEFINE` es una directiva muy util. Define se usa para crear pequeñas macros. Con estas macros podremos poner nombres a pequeños fragmentos de código que nos facilitarán la realización y comprensión del algoritmo.

Por ejemplo, podremos poner nombres a bits.

```
#define CERO STATUS,2
```

Así, en vez de tener que llamar al bit por un numero y un registro, podremos usar directamente la palabra **CERO**.

```
#define CINCO 5
```

Cada vez que se utilice la palabra **CINCO** será reemplazada en el momento de la compilación por el número 5.

Otro ejemplo muy práctico es el de poner nombre a un fragmento de código usado frecuentemente. Este fragmento de código, puede ser por ejemplo, el que conmuta entre los dos bancos.

```
BSF    OPTION,RP0
BCF    OPTION,RP0
```

Como cambiamos varias veces de banco a lo largo de un algoritmo, puede resultar más práctico ponerle un nombre.

```
#define BANCO1 BSF OPTION,RP0
#define BANCO0 BCF OPTION,RP0
```

De este modo bastará con poner **BANCO1** o **BANCO0** para conmutar entre los dos bancos de memoria de manera que cada vez que se utilice la palabra **BANCO1**, en realidad se estará utilizando la instrucción **BSF STATUS,RPO**

En el siguiente ejemplo:

```
#define salida PORTA,3
```

No tendremos necesidad de recordar cual era la patilla de salida, sino que solo lo mencionaremos como **salida**. Cada vez que aparezca la palabra **salida** en el código, ésta será interpretada como PORTA,1 que es una instrucción válida. Podemos ponerlo a cero con la instrucción.

```
BCF salida
```

En vez de tener que poner.

```
BCF PORTA,3
```

Una cosa a tener en cuenta es que con la directiva INCLUDE, podemos prescindir del carácter almohadilla (#), pero en el caso de la directiva DEFINE, no.

Esta directiva es muy util porque hace el código más fácil de leer y entender.

Directiva TITLE

Esta directiva no sirve de mucho, pero será útil para aquellos que quieran que el compilador tenga en cuenta el título que le ha puesto a su código. Tiene la siguiente sintaxis:

```
TITLE "Nombre del código"
```

Este nombre aparecerá en los archivos .lst (listados) que cree el compilador.

Directivas IF...ELSE...ENDIF

Algunos ensambladores permiten incluir o excluir partes del programa dependiendo de condiciones que existan en el tiempo de compilación.

La forma típica es:

```
IF CONDICION
    .
    .
ELSE
    .
    .
```



```
ENDIF
```

Ejemplo:

```
SINK EQU 1 ; (cambiar por 0 en caso necesario)
IF SINK=1
    BCF PORTA,0
ELSE
    BSF PORTA,0
ENDIF
```

En este caso el valor de **SINK** hará que el compilador utilice distintas instrucciones de código.

Si la condición es verdadera en el tiempo de compilación, las instrucciones que están entre IF y ELSE se incluirán en el programa. Si la condición es falsa se incluirán en el programa las instrucciones entre ELSE y ENDIF.

Los usos típicos son:

- Para incluir o excluir variables extras
- Para incluir código de diagnóstico en condiciones de testeo (DEBUG).
- Para permitir datos de distintos tamaños.

Desgraciadamente, el ensamblado condicional, tiende a complicar la lectura del programa, por lo tanto, sólo debemos utilizarlo si es necesario.

Directiva MACRO

Esta directiva resulta muy potente y a diferencia de la directiva #define se pueden crear macros más extensas, lo que nos evitará tener que ejecutar reiteradamente fragmentos de código idénticos. Cuando una macro es invocada, esta es copiada por el ensamblador en el lugar de la invocación dentro del código fuente. La macro se declara con la directiva MACRO, y termina con la directiva ENDM.

Creación de una macro denominada **activar**:

```
activar macro
    CLRF PORTA
    BSF PORTB,2
endm
```

Hemos creado una macro llamada **activar** de manera que en nuestro código cada vez que pongamos la palabra **activar**, el ensamblador la reemplazará por **CLRF PORTA...** etc. hasta el final de la macro que termina con la directiva **ENDM** (fin macro).

Las macros permiten asignar un nombre a una secuencia de instrucciones de manera que son útiles cuando ocurren secuencias de instrucciones repetitivas. Luego se utiliza el nombre de la macro en el programa como si se usara la secuencia de instrucciones anterior.

Las macros no son lo mismo que las subrutinas. El código de las subrutinas aparece una sola vez en un programa y la ejecución del programa salta a la subrutina. En cambio, el ensamblador reemplaza cada aparición del nombre de la macro con la secuencia especificada de instrucciones. Por consiguiente la ejecución del programa no salta a la macro como una subrutina.

Ejemplo:

Archivo "MULX10.ASM"

```
MULX10  MACRO                ;comienzo de la macro
        MOVF    tiempo,W    ;guarda el tiempo en W
        RLF     tiempo      ;multiplica por 2
        RLF     tiempo      ;multiplica por 2
        RLF     tiempo      ;multiplica por 2
        ADDWF   tiempo      ;le suma una vez más
        ADDWF   tiempo      ;le suma una vez más
        ENDM                ;fin de la macro
```

Archivo "EJEMPLO1.ASM":

```
#INCLUDE    "MULX8.ASM"
tiempo      EQU    0Ch
resultado   EQU    0Dh

        MOVLW   20
        MOVWF   tiempo
        MULX10
```

```
MOVWF resultado
END
```

Si ensamblamos "EJEMPLO1.ASM" notaremos que el listado final (EJEMPLO.LST) queda de la siguiente forma:

```
tiempo      EQU    0Ch
resultado    EQU    0Dh

            MOVLW    20
            MOVWF    tiempo
            MOVF     tiempo,W    ;guarda el tiempo en W
            RLF      tiempo      ;multiplica por 2
            RLF      tiempo      ;multiplica por 2
            RLF      tiempo      ;multiplica por 2
            ADDWF    tiempo      ;le suma una vez más
            ADDWF    tiempo      ;le suma una vez más
            MOVWF    resultado

END
```

Problemas con las MACROS

Con las macros se puede trabajar rápidamente, pero pueden resultar poco eficientes. Veamos un error muy común al utilizar macros, en este caso se utiliza una macro denominada MOVFF:

```
MULX10      MACRO                ;comienzo de la macro
            MOVF     AUX1,W      ;Mueve contenido de un registro a otro
            MOVWF    AUX2        ;a través del acumulador
            ENDM                ;fin de la macro
```

Porción de código:

```
MOVLW .1          ;TEMP=1
MOVWF TEMP
DECF  TEMP,F      ;Z se va a 1
BTFSS STATUS,Z    ;salta si o si
MOVFF AUX1,AUX2   ;Macro
MOVWF PORTA
; ...
```

En la línea de la macro está el error porque los saltos (BTFSS) no pueden saltar macros. Las macros están compuestas por más de una instrucción, y el salto se producirá dentro de la misma.

El código anterior con la macro incrustada sería:

```

MOVLW .1          ;TEMP=1
MOVWF TEMP
DECF  TEMP,F      ;Z se va a 1
BTFSS STATUS,Z    ;salta si o si
MOVF  AUX1,W      ;líneas de anterior macro
MOVWF AUX2        ;
MOVWF  PORTA
; ...

```

Otro tema importante, que se ilustra en este ejemplo, es que las macros pueden modificar registros (en este caso W) de forma que el programador podría no tener en cuenta.

En el ejemplo anterior, PORTA se debería cargar con 1, que aparentemente era el valor de W, pero la macro lo ha modificado, lo que resulta en otro error.

Ejemplos de macros

```

; *****
; macros.asm ;
; "MACROS para 16F84" ;
; *****

callz    macro    subrutina
          btfsc    STATUS,Z
          call     subrutina
          endm

callnz    macro    subrutina
          btfss    STATUS,Z
          call     subrutina
          endm

movff     macro    f2,f1 ;(atención, se destruye W)
          movf     f1,w
          movwf    f2
          endm

movlf     macro    file,literal ;(atención, se destruye W)
          movlw    literal
          movwf    file
          endm

;Atención, para usar estas macros ya debe estar activo el banco 1
CONF_PORTA macro    dato
            movlw    dato
            movwf    TRISA
            endm

CONF_PORTB macro    dato
            movlw    dato
            movwf    TRISB

```

```

        endm

;configurar Option Register:
CONF_OPTION macro dato
        movlw dato
        movwf OPTION_REG
        endm

;configurar el registro de interrupciones:
CONF_INTCON macro dato
        movlw dato
        movwf INTCON
        endm

SET_BANK_0 macro
        BCF STATUS,RP0
        endm

SET_BANK_1 macro
        BSF STATUS,RP0
        endm

;enable y disable all the mascarable interrupts (16F84):
EI macro
        bsf INTCON,GIE
        endm

DI macro
        bcf INTCON,GIE
        endm

#define iEnable EI
#define iDisable DI

;arrancar el timer:
RESET_TIMER macro
        bcf INTCON,T0IF
        endm

; inicializar timer antes de hacer RESET_TIMER para que arranque.
INIT_TIMER macro dato
        movlw dato
        movwf TMR0
        endm

jmp macro salto
        goto salto
        endm

ret macro
        return
        endm

;Complemento a 1 de W:
comw macro
        xorlw 0xff
        endm

```

;Instrucciones de salto tipo Z80

```
jz      macro  _salto  ;salta si zero
        btfs   STATUS,Z
        goto   _salto
        endm
```

```
jnz     macro  _salto  ;salta si no zero
        btfs   STATUS,Z
        goto   _salto
        endm
```

```
jc      macro  _salto  ;salta si carry
        btfs   STATUS,C
        goto   _salto
        endm
```

```
jnc     macro  _salto  ;salta si no carry
        btfs   STATUS,C
        goto   _salto
        endm
```

```
; *****
;      FIN
; *****
```